

Q-Framework Evidence Whitepaper v1.0

RC3

検証可能な異種AIコンピューティング継続アーキテクチャ

公開スナップショット：2026年10月7日

Q-Framework は、実際に開発・運用・検証が進められている独自の異種AIコンピューティング・アーキテクチャです。

中心となる問いは、複数GPUを「物理的に1枚の巨大GPUとして扱えるか」ではありません。

実行中のAI workloadは、必ずしも1枚のGPU、1つのprocess、1つのnodeに恒久的に縛られないといけないのか？

Q-Framework は、compute state、continuation、heterogeneous workers、reference-state consistency、cross-boundary execution について、実運用に近い検証結果を積み上げています。

RC3 では、公開方針を明確にしています。

半分は真実、半分はブラックボックス。

「真実」とは、公開する測定値、SHA、bitwise comparison、numerical comparison、完成出力、ハードウェア級別がすべて実際の検証記録に基づくことを意味します。

「ブラックボックス」とは、production engineを再構築できる内部state schema、アルゴリズム、制御式、係数、restore sequencing、scheduler policy、source codeを公開しないことを意味します。

0. 用語の明確化：QmRNA の Messenger コンセプト

- QmRNA は Q-Framework が定義した runtime messenger / control-signal layer です。この名称は、生物学の mRNA が「指令を運ぶ messenger」であるという概念を意図的に借用しています。Q-Framework では、QmRNA が workload intent と control signal を QVRAM / VRAM-facing runtime に届け、execution layer が residency、feed、checkpoint、rescue、resume などの状態を扱えるようにします。
- これは計算アーキテクチャ上の比喩と命名思想であり、生物 RNA、ワクチン、遺伝子工学の実装ではありません。Q は Quantum を意味しません。
- 本プロジェクトは量子計算フレームワークではありません。
- H3 は MiniMax H3 AI video-generation workload / model path を指します。

- **OOM** は CUDA / PyTorch / GPU VRAM 文脈の **Out of Memory** を意味します。
- **Recovery Path** は AI compute-state の checkpoint / restore / continuation を指します。

Canonical interpretation: QmRNA = Q-Framework messenger control-signal layer — inspired by the messenger role of mRNA, implemented as an AI/GPU runtime abstraction.

1. なぜQ-Frameworkを作るのか

大規模な生成AI workloadでは、単一のexecution resourceがボトルネックになりやすく、VRAM pressure、process interruption、worker availability、長時間ジョブなどによって、すでに計算した作業を捨てて再計算するケースがあります。

Q-Framework が探っているのは別のシステムモデルです。

- すでに完了した計算結果を、検証可能な形で残せるか
- workloadを別のexecution resource上で継続できるか
- 共通のidentity/reference authorityを複数workerで共有できるか
- 異なるnode/runtimeが同じ数値状態から一致する結果を再現できるか
- 長時間workloadを一時停止し、保存し、あとで再開できるか
- 独立した複数GPUが1つのexecution lineageに参加できるか

2. これは監視システムではない：OOM制御は実行経路そのものに組み込まれている

公開されている **Live Compute** ページは、Q-Framework の観測・証拠インターフェースです。GPU / VRAM、QmRNA、QSTATE、branch progress、node heartbeat、runtime progress を表示しますが、**telemetry 自体が Q-Framework の中核機能ではありません**。

production path は、実際の MiniMax H3 / ComfyUI 系 workload の execution lifecycle に介入します。公開可能な高レベル動作には以下が含まれます。

- VRAM pressure / PRE-OOM 条件で recoverable checkpoint / QSTATE / QREV を作成する；
- 保持すべき computation state を CPU RAM または SSD backing store へ spill する；
- 不要な GPU residency を解放し、利用可能な VRAM を回復する；
- 保存した state を同じ GPU に restore して **same-GPU continuation** を行う；
- または別の GPU worker に restore して **mixed-GPU continuation / rescue** を行う；
- restore 後に sampler / model execution を継続し、workload 全体を最初から再計算しない；
- receipt、SHA、numerical evidence により continuation lineage を検証する。

したがって目的は「OOM を監視する」ことではありません。memory pressure の前後で workload lifecycle を変え、可能な限り有効な計算状態を保存し、residency を解放し、state を復元して実行を継続することです。

これは単純な VRAM monitor/debug node とは異なります。また empty_cache()、解像度低下、量子化、attention chunk size の縮小だけと同義でもありません。それらは局所的な最適化として併用可能ですが、Q-Framework が扱うのはより上位の **stateful memory-pressure recovery + resumable execution** です。

公開証拠には、実 H3 continuation、cross-GPU restore 後の completed output、約 250 MiB state transport の bitwise integrity、multi-worker execution lineage が含まれます。production pressure scoring、state packing、residency / eviction policy、restore sequencing、model-forward modification はブラックボックスのままです。

Q-Framework は OOM-aware execution / recovery implementation であり、OOM telemetry dashboard ではありません。

ただし claim discipline は維持します。これは「どのモデルでも絶対に OOM しない」「zero OOM probability」を意味しません。正確な主張は、**VRAM pressure / OOM recovery の場面で、実際の AI workload state を保存・解放・復元・継続する execution path を実装し検証している**ということです。

ComfyUIへの実装位置：外部クラウドから監視しているだけではない

現在の H3 recovery path は、**Windows 上のローカル ComfyUI runtime に実際に統合されています**。外部のクラウドサービスが ComfyUI を監視しているだけではありません。

公開可能な非再構築レベルでは、統合は次の4層です。

1. **ComfyUI process 内の node-local runtime hook / custom-node layer** が QmRNA / QVRAM control state を受け取り、checkpoint を作成し、PRE-OOM / HARD-OOM recovery contract に参加し、検証済み H3 path では対応する low-memory attention / paging candidate を選択する。
2. **Resilient submit / Master layer** が Job seed、lineage、QSTATE、QREV、worker ownership、recovery receipt を維持し、same-GPU re-arm または mixed-GPU rescue を制御する。
3. **Bridge / backing-store layer** が CPU RAM / SSD backing、cold-store、restore、cross-worker continuation を担当する。
4. **Live Compute** は上記の実行状態と証拠を表示するだけで、recovery engine そのものではない。

つまり Q-Framework は、ComfyUI を別の外部 scheduler に置き換えて OOM 解決を主張するものではありません。ComfyUI は引き続き model graph / sampler を実行し、その execution path の内外に node-local runtime control、state checkpoint / restore、lineage orchestration を追加します。

また、現時点では **ComfyUI Manager からワンクリックで導入できる汎用 plugin 製品ではありません**。production implementation は、検証に使用している ComfyUI + MiniMax H3 runtime に統合されています。一般ユーザー向け installer 化は Experimental Preview / productization の別工程です。

統合モデル：Extension Layer であり、ComfyUI Core の「魔改」ではない

Q-Framework と ComfyUI の正式な関係は **runtime integration / extension layer** です。Q-Framework は ComfyUI core fork として定義されておらず、検証済み path では公式 source tree を private modified distribution に置き換えることを前提にしていません。

現在検証済みの Windows + H3 path では：

- node-local integration は ComfyUI の **custom-node / runtime-extension layer** に配置される；
- recovery、QmRNA gate、checkpoint / restore は Q-Framework integration component が担当する；
- Master、Bridge、QSEARCH、QSTATE / QREV lineage orchestration は ComfyUI process 外部に存在する；
- 検証済み low-memory attention candidate は **controlled model instance / execution path** に適用され、ComfyUI の global backend を恒久的に置き換えない；
- 検証済み path は Q-Framework が ComfyUI の execution.py、model_management.py、または公式 source tree 全体を置換することを要求しない。

したがって「core source の魔改」「作者独自の ComfyUI fork」「作者の fork を入れないと動かない」という表現は、この architecture の正確な説明ではありません。

正確な表現は：

Q-Framework integrates with ComfyUI through a node-local runtime extension plus external state/orchestration layers; it does not define itself as a fork of ComfyUI core.

この設計は ComfyUI core file への直接依存を減らすことを意図していますが、将来のすべての ComfyUI version に自動互換であることを意味しません。対応 version / model path ごとに検証が必要です。

過大解釈を防ぐ境界

RC3 は以下を主張しません。

- 任意の PyTorch operator の次の contiguous VRAM allocation を常に正確に予測できること。
- 任意モデル／任意 Attention を OOM 前に自動で chunked execution へ書き換えられること。
- ComfyUI native memory manager を完全に置き換えること。

検証済みの主張は、対応する H3 execution path において、QmRNA / QVRAM / recovery layer が checkpoint、spill / release / restore、same-GPU / mixed-GPU continuation を行い、対応する検証済み candidate では low-memory attention / page-execution control を使用できる、というものです。

つまり、実際に動作した **recovery capability** を公開し、実験中の内部機構を **universal PyTorch feature** として一般化しません。

名称の明確化

本プロジェクトは Unity QFramework、NVIDIA CUDA-Q、その他の同名・類似名 framework とは技術的な所属関係がありません。本 repository における **Q-Framework** は、ここで記述する heterogeneous AI state virtualization、OOM-aware recovery、compute continuation architecture を指します。

3. 4つの検証済み能力

2.1 Mixed-GPU Compute Continuation

20-step deterministic test では、control path と continuation path の最終結果が以下の通り一致しました。

- bitwise equality : **true**
- allclose : **true**
- max absolute difference : **0.0**
- mean absolute difference : **0.0**
- SHA-256 : ae04c23d4daed035632855260f85c360a857c28a8d46544f5c92f91baff823a3

重要なのは、単にデータを移動できたという点ではなく、継続後の最終計算結果が、検証条件下で完全一致したことです。

2.2 Same-Worker / Same-GPU Continuation

Q-Framework は、常に cross-GPU migration を必要とするわけではありません。

同じ worker/GPU を継続利用できる場合には、固定 owner 型の continuation も検証されています。

checkpoint 構造、locality policy、resume sequencing の内部仕様は非公開です。

2.3 H3 Memory Static / Shared Reference-State Compute

異なる execution worker が共通の authoritative reference state を基準に処理する経路も検証されています。

Cross-node reference-state validation では、同一の SHA-256 が得られました。

1488ccf08323200643d5759046f2d47e1e16d44a70f278a79346aec53103d937

これは、少なくとも検証条件下では、異なる node 間で共通 reference から再現可能な計算状態を形成できることを示しています。

2.4 Mixed-GPU Cross-Boundary Compute

実際の長尺映像や複雑なAI workloadは、1つのartifactやshotだけで完結するとは限りません。

Q-Frameworkでは、GPU worker、node、media/artifact boundaryをまたぐexecution lineageが検証されています。

記録済みのsmoke output SHA-256：

7A909C59636C1A18C6B1A1C7CD2037120F25DDC2C7F36CD2E62B4C70A43212E1

後続のcompleted artifact：

C35FAECFE3C1BD0B5338CC4BE0EB8A854A10CE0FEEF86D39CA750DE6BB4A357C

Boundary representationおよびcontinuity contractはブラックボックスです。

4. 約250 MiBのState Transport検証

約**250 MiB**のfrozen computation objectについて、transport integrity testを実施しました。

結果：

- source / staged / restored read-back SHAが完全一致
- SHA-256： 330674e23bf109a4f5e3983608ce118274373deae669e3d3bacee2f77639d5a6
- bitwise SHA match：**true**
- restore前にsource execution allocationを解放
- end-to-end measured test time：約**3.675秒**

これはtransport integrityの証拠であり、任意モデルや任意stateへの一般化を意味するものではありません。

5. Controlled Proofから実H3 workloadへ

Q-Frameworkはsynthetic state testだけでは終わっていません。

実際のH3 workloadで、別GPU workerへのcontinuation後にmedia artifactの生成を完了しています。

- 480 × 832
- 24 fps
- 39 frames
- output SHA-256： 307828DE5D0D54F36004EF9C97B2E4DAE5178E59A9C8EF74775F07522B788AB3

これは、「stateが保持できるか」から「実モデルのworkloadが継続し、出力まで完了できるか」へ検証段階を進めた結果です。

6. 4枚の独立22GB GPUによるproduction-class test

4枚の独立したRTX 2080 Ti 22 GB workerが、同一のproduction-class execution lineageに参加しました。

最終artifact：

- duration：**15.000秒**
- frames：**540**
- participating GPU workers：**4**
- final SHA-256：23CDBA13DBFF7B8EC234DB1D7DE61625597E128BD6684849FCB6CD33650ADF57

重要な点：

Q-Frameworkは、4枚の22GB GPUが物理的に1枚の88GB GPUになるとは主張していません。

検証しているのは、複数の独立GPU workerが同一の継続可能なexecution体系に参加できることです。

7. Node + GPU：Cross-Environment Numerical Consistency

production-derived numerical taskを、3つの異なるexecution environmentで独立再計算しました。

3環境すべてで同じrecomputed-result SHA：

330271d14ee7086e27f875a4a56e720f169cdef1523e908ad94696bb70df15a4

production sourceに対するfloat32 reconstruction comparison：

- max absolute error：4.76837158203125e-07
- mean absolute error：約2.03e-08

Exact hash一致と浮動小数点近似は、異なる証拠として明確に分けています。

8. QSTATIC + QmRNA Controlled Equivalence

同じreference、同じseed、同じgeneration conditionsでcontrolled A/B testを行いました。

MP4 container hashは異なりましたが、decoded visual contentを比較すると：

- decoded frames : **22**
- exact matching frame hashes : **22 / 22**
- framemd5 manifest SHA-256 :
FF55120ADF80F99D74405C5DECA7A9B54116B01F987591A012BD1FF01467F5D7

このcontrolled runでは、alternate runtime/control pathを使ってもdecoded visual resultが一致しました。

QmRNAの内部表現、QSTATIC contract、production control pathは非公開です。

9. SHA証拠の意味

Q-Frameworkでは、SHAをすべて同じ意味で扱いません。

主に以下を区別します。

1. **Transport Integrity** — 同一のfrozen objectが移送後も完全一致するか
2. **Computed Result Consistency** — 異なるexecution environmentで同一の計算結果が得られるか
3. **Decoded Output Equivalence** — containerが異なってもdecoded frameが一致するか

これにより、「見た目が似ている」と「検証可能に一致している」を分離できます。

10. Q-Frameworkが本当に変えたいもの

従来のAI infrastructureでは、GPUがworkloadの中心であり続けることが前提になりがちです。

Q-Frameworkは別の仮説を検証しています。

GPUはexecution workerであり、workload lifecycleの恒久的ownerである必要はない。

この仮説が、より長く、より異種で、より複雑なworkloadで成立し続けるなら、以下に影響する可能性があります。

- consumer GPU utilization
- mixed-generation GPU fleet
- edge / workstation AI
- interrupted workload recovery
- long-running generative workloads
- heterogeneous compute economics

これらは研究・製品化の方向性であり、未検証の性能保証ではありません。

11. 今後の検証段階

RC3は完成版ではありません。

Q-Frameworkの開発は「成功／失敗」の二分法では捉えていません。

1つの難題を解けば、次のより難しい条件が現れる。

現在の主な検証項目：

Challenge A — Longer Continuous Workload

より長い生成workloadへverified continuationを拡張する。

Challenge B — Wider Heterogeneous Fleet

より多くのGPU generation、driver、runtime、node combinationを検証する。

Challenge C — Persist Now, Resume Later

workloadを意図的にpauseし、persistし、後でcontinuationできるようにする。

Challenge D — Time-to-Result

「できる」だけでなく、wasted recomputationを減らし、end-to-end completion timeを短縮する。

Challenge E — Repeatability at Scale

より多くのrun、worker、node、hardwareで同一クラスのproofを再現する。

12. 公開進捗と今後のマイルストーン

今後Q-Frameworkが新しい技術課題を解決するたびに、少なくとも以下のような新しい検証可能情報を公開する予定です。

- completed artifact
- new SHA receipt
- longer duration
- more participating workers
- new hardware/runtime combination
- repeat-run statistics
- pause/persist/resume evidence

- worker takeover evidence
- before/after time-to-result
- new QSTATIC / QmRNA equivalence result

本whitepaperは最終発表ではなく、今後の検証結果・測定値・公開可能な証拠に応じて更新される技術記録です。

13. Experimental Preview

Q-Frameworkは、外部の研究者、開発者、協力候補が一部の検証済み能力を制限された条件下で評価できる **Experimental Preview** の公開を予定しています。

Previewは完全なproduction engineではありません。

候補となる機能：

- limited continuation experiment
- pause / persist / resume experiment
- SHA / receipt verification
- reference-state consistency experiment
- selected heterogeneous-worker demonstration

Previewは封装・制限・ブラックボックス方式で提供し、核心アルゴリズムとproduction implementationは公開しません。

将来的には：

Experimental Preview — Build #001 / #002 / #003 ...

という形式で、各buildごとに公開検証範囲と機能成熟度を段階的に拡張する方式を想定しています。

14. 公開とブラックボックスの境界

公開

- 実測benchmark result
- output / proof hash
- bitwise / numerical comparison
- sanitized hardware class
- workload metadata
- completed artifact evidence
- milestone / challenge progress

ブラックボックス

- production state schema
- state serialization / packing
- restore sequencing
- QmRNA signal representation
- QmRNA control equation / coefficients
- QSTATIC internal contracts
- QVRAM residency implementation
- memory-pressure threshold
- eviction / prefetch policy
- scheduler / worker-selection
- lease / fencing / retry implementation
- production model-forward modifications
- private topology / endpoints
- proprietary source code

ブラックボックスは偽データを意味しません。

証拠は真実でなければならない。実装をすべて公開する必要はない。

15. Claim Discipline

Q-Frameworkは現時点で以下を検証済み結論としては主張しません。

- unlimited VRAM
- zero OOM probability
- universal model compatibility
- universal bitwise determinism
- 複数GPUの物理的VRAM統合
- benchmarkのないuniversal acceleration

Claimは常にevidenceの後に続きます。

16. 結論

Q-Frameworkで重要なのは、単一のSHAや単発の動画成功ではありません。

徐々に形成されているevidence chainは：

verifiable state → result-preserving continuation → real H3 continuation → multi-GPU participation → cross-environment consistency → cross-node reference consistency → controlled runtime-path output equivalence

次の目標は、このevidence chainをさらに長く、速く、異種化し、再現性を高めることです。

production mechanismは引き続きブラックボックスです。

Real evidence. Black-box mechanism. 今後の進展は検証可能な結果に基づいて公開します。